

Extended Abstract

Background In the semiconductor industry, metrology and inspection tools are essential for wafer fabrication process control, performing precise measurements of silicon wafers at various target locations. These measurements can be conducted under different optical conditions, such as varying light wavelengths, intensity, polarization, apodization and focus positions. Each set of optical conditions is referred to as "recipe". A "recipe" stands for a "list of configurations" that should yield the best accuracy and precision possible.

Motivation Selecting the optimal configurations for each wafer layer fabrication process is crucial for accurate measurements. Currently, this selection process involves a lengthy and costly semi-automatic procedure that compares and maps the measurement performance of numerous optical configurations.

Method This project explores two reinforcement learning (RL) methods to optimize recipe selection in semiconductor metrology. The first approach, Vanilla Q-Learning, serves as a baseline to evaluate RL's effectiveness in navigating the configuration landscape. By limiting the action space to simple navigation movements across a 2D grid defined by wavelength and focus position, the method addresses the curse of dimensionality. The reward system is designed to guide the agent towards optimal configurations while penalizing failures, incorporating KPI-based rewards to enhance exploration efficiency.

The second approach, Model-Based Sampling Optimization with Ensemble-Based Uncertainty Estimation, leverages an ensemble of neural networks to iteratively refine a value function. This method actively samples configurations, estimating values and uncertainties to guide selection. Starting with a sparse subset of samples, the ensemble model scores unsampled states, selects top candidates, queries true values, augments the dataset, and re-trains iteratively.

Implementation The entire code base was implemented in Python. For the Q-Learning algorithm, the environment was built according to the dataset in hand. A reward function was then populated each one of the cells on the grid against the KPI that is being assessed, and the Q-table was initiated with zeros. The training process then was executed based on several tunable parameters. The evaluation is based on the success rate of the agent navigating to the goal from a set of random starting points. As for the second algorithm, the neural network ensemble was initially trained on a sparse sampling data and then entered an iterative process of selecting the top candidates based on a weighted evaluation of exploration vs exploitation, where in each iteration the ensemble was re-trained using the new data.

Results After a fine tuning of the Q-learning hyperparameters, the algorithm achieves 100% success rate and retains this performance on other datasets. The model-based sampling optimization yields very good prediction errors using only a fraction of the original dataset and shows good generalization across multiple datasets.

Conclusion Overall, both algorithms achieve good performance and generalization across multiple datasets. Vanilla Q-Learning shows 100% success rate but it is not applicable to industry application as the reward function is not known in advance. On the other hand, model-based sampling optimization advantage is its capability to predict the reward function that is based on initial sparse sampling and thus addresses the challenges that are associated with the traditional "grid search" approach for recipe optimization.

Optimizing Optical Configuration Selection for Semiconductor Metrology with RL

Yatir Linden
GCOE student
mans3579@stanford.edu

Ofer Manos
GCOE student
sum1elf@stanford.edu

Abstract

In the semiconductor industry, every fabrication process step usually entails choosing the best performing configuration that will yield the best accuracy and precision out of an intractable number of such configurations. In optical metrology, overlay is one of the critical key performance indicators that determine the yield of the final chips; however, choosing the best performing configuration relies on old, brute-force, approach that is lengthy and challenging. To overcome these limitations, this work explores the option of using RL methods to predict the best configuration while using the least number of sampled configurations. In particular, attempts to harness vanilla Q-Learning and Model-based sampling optimization were executed. The former algorithm shows excellent success, while the latter yields promising results by being able to predict the reward function in advance on multiple datasets. Certainly, Q-Learning is not applicable for real-world application; but, the model-based sampling optimization has far-reaching implications in terms of reducing tools cost-of-ownership without compromising the traditional, "grid search", approach performance.

1 Introduction

In the semiconductor industry, every fabrication process step is monitored using dedicated tools to ensure that the tight specs are being met May and Spanos (2006). One of the most important metrics in the industry is called overlay (OVL), which is a measure of misregistration between previous and current fabrication layers Katz et al. (2022). OVL has a great impact on the yield of the final devices, and is measured on hundreds of target locations, dubbed "sites", across the wafer. OVL measurements can be conducted using different optical conditions, such as varying light wavelengths, intensity, polarization, apodization, and focus positions. Each set of optical conditions is referred to as "recipe". A "recipe" stands for a "list of configurations" that should yield the best accuracy and precision possible.

The complexity of the metrology tools has been increasing and forces the user to try to select the best performing recipe from an infinitely large number of possible configurations Kim et al. (2021). As manual recipe selection is impractical, an "automatic recipe optimization", ARO, is widely adopted in the industry. In ARO, the user chooses selected configurations that are of interest as well as their sampling range and intervals. The ARO sequence is not more than a "brute-force" approach aiming to cover all hardware configurations. It is usually done in two main cycles (see Figure.1), the first cycle aims to measure as many configurations as possible on a minimum number of measurement locations, sites. The main goal of ARO cycle one is to quickly filter out poor optical conditions, e.g. conditions that result in defocused images, low visibility of previous layer, or poor contrast. Then, cycle two focuses on the good configurations that were left and performing dozens of measurements for every condition, to increase the selection confidence of the top-k recipes. The final ranking of the recipes is assessed by several key performance indica-

tors, KPIs, to determine the recipe accuracy and precision as well as its robustness to process variation.

Given the fact that a typical product line contains hundreds of even thousands of processing steps, where each one of them has a dedicated recipe, a faster recipe selection procedure has the potential of saving millions of dollars in lost cost-of-ownership and production yield. Thus, the current project aims to explore modern reinforcement learning (RL) algorithms to improve the efficiency of the brute-force approach for recipe selection while retaining or improving measurement accuracy and stability.

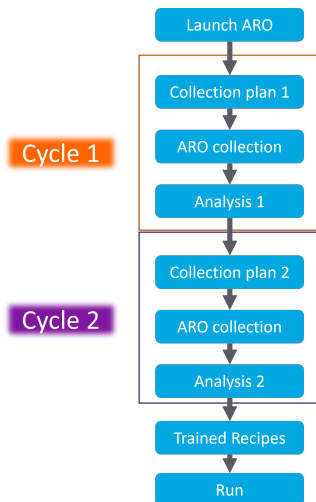


Figure 1: flow chart shows the ARO process divided into two cycles.

2 Related Work

As the ARO process is costly in terms of cost-of-ownership (usually referred to as in the industry as time-to-recipe), several attempts have been made to optimize this process. Kim et.al. Kim et al. (2021) introduced a method called Statistical Process Optimization & Control (SPOC) that relies on statistical tools to detect the best candidates within a narrow range of values; however, this still requires sparse sampling of the entire configuration space. Nevertheless, the authors argue that using this method can shorten the time-to-recipe from about 10 hours down to 2.

A very recent publication by Chen, et.al. Chen et al. (2025), uses meta-learning RL to optimize recipe configuration for chemical vapor deposition (CVD) process. CVD recipes are similar to OVL recipes in a sense of their intractable number of available parameters that poses challenges on choosing the best recipe in the shortest time possible. The authors point out that their algorithm leverages limited historical data and yields good results; however, it comes with a set of challenges such as computational cost, recipe selection bias, and performance deterioration due to convergence to the cost function local minimum. Han and co-workers Han and May (1997) have demonstrated recipe selection optimization using neural networks in plasma enhanced chemical vapor deposition (PECVD) process. Their experimental results show the superiority of this approach vs the traditional algorithm. It is important to note however that PECVD recipe selection could be very challenging due to its parameter space that includes several continuous variables with complex relationships to one another.

On a wider perspective, RL has not got much traction in the semiconductor industry. As a side note, it is worth mentioning that machine learning more generally has gradually started penetrating this conservative industry (the reader is referred to following examples and the citations within: Ophir et al. (2021), Verner et al. (2020), Shusterman et al. (2023)). Nevertheless, there have been several attempts to harness RL mainly for run-to-run control Fan and Chen (2025) Liu et al. (2025) Ma and Pan (2023). Run-to-run control is a feedback loop that uses data to continuously calibrate different apparatuses in order to maintain process stability. The lack of published research is

perhaps also due to intellectual property (IP) restrictions, and we have no doubts that this will be a hot topic in the foreseeable future.

3 Method

For this project, two RL methods have been applied. The first is vanilla Q-Learning to assess the feasibility of RL to address the recipe selection challenge as well as to serve as a baseline algorithm. The second method is "Model-Based Sampling Optimization with Ensemble-Based Uncertainty Estimation" to assess the performance for a real industry application.

3.1 Vanilla Q-Learning

Q-Learning Watkins and Dayan (1992) is notoriously known to suffer from the *curse of dimensionality* as it is unmanageable to construct a Q-table based on a high-dimensional state and/or action space. In our case, and provided that we rely only on the data we possess without interpolation, the state space is not that large; however, the action space can be very large if the agent can sample any configuration at every given state. In order to limit the action space, this problem can be thought of as a simple navigation problem where the agent is restricted to move one step, up/down or right/left to sample the next configuration.

The state space of the agent spans to 2D space where one dimension is wavelength and the other is focus position. The action space, as stated, consists of 4 dimensions. The reward function was constructed deliberately to encourage the agent to navigate from a starting location, i.e. state, to the goal with the least number of steps. The agent receives -100 reward if it steps on a configuration (state) that was failed, or moves outside of the limits of the configuration landscape; both are terminal states. Other than that, the agent receives -1 when moving to any state except for the goal. Lastly, the goal, i.e. the state in which the best recipe configuration resides, gets a reward of $+100$.

The aforementioned is the basic environment configuration, but other attempts were made in order to increase the success rate. Below are the possible environments setups and tunable training parameters and the logic behind them:

- **Reward function:** The obvious choice as detailed above is assigning a homogenous reward of -1 when stepping on a non-terminal state. However, another option is to encourage the agent to move in a gradient-ascent manner by assigning a less-negative reward when the KPI value is higher. Theoretically, the agent should opt to go "up and down" the configuration landscape to search for the highest peak.
- **Dynamic reward function:** Although the agent is encouraged to find the highest reward and refrain from moving to terminal states, a small tweak might nudge the agent to reach the goal faster, i.e. using dynamic reward function. The logic behind this is that the agent should not visit the same state twice and should keep exploring the environment. To address this, a dynamic reward function was constructed in the training phase to give a very large negative reward to every state that was already visited during an episode.
- **Explore all starting points:** As the state space is quite large and consists of about a thousand states, it is statistically possible that not all states will be visited in a full training phase for several hundreds of episodes and thus leaving the q-table with its initial values. To overcome this problem, an option was added to start from a random state that has not been visited during the current episode.

A pictorial illustration of the Q-Learning setup is shown in the figure below.

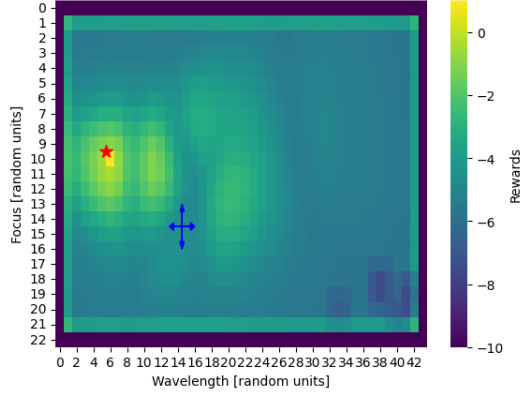


Figure 2: An illustration of Q-Learning reward map based on the values of the desired KPI. It shows the terminal states with reward of -10 , the goal with reward $+1$, and all the other non-terminal states get negative values as a function of the KPI at that state. A random starting point is shown as well where the agent should decide where to move (blue arrows) in order to navigate to the goal that is marked as a red star. For visibility, the reward function is convoluted with 3 by 3 kernel and the terminal states values were set to less positive and negative values.

3.2 Model-Based Sampling Optimization with Ensemble-Based Uncertainty Estimation

This sample-efficient model-based reinforcement learning approach iteratively refines a value function over the state space using active sampling. The method leverages an ensemble of neural networks to estimate both the value and uncertainty, guiding the selection of new samples.

Models

- **Input:** The embedded optical configuration state constructed from both discrete features (such as Polarizer P/S, Aperture Low/Mid/High and Direction X/Y) and continuous features (Wavelength and Focus offset).
- **Output:** Set of measurement metrics (Inner/Outer Harmonic detectivity) used as a value estimation.
- **Initialization:** Each NN model is initialized with different random weights.
- **Ensemble:** An ensemble of the N models is constructed to estimate the value μ (Mean) and uncertainty σ (Stddev) for an input state (Figure 3).

Sequence

- **Initial Dataset:** Sparse subset grid of the state space is sampled, in a single wafer location (site).
- **Initial Train:** All models in the ensemble are trained on the initial dataset.
- **Sampling Loop:** Repeat until convergence or budget is exhausted - for each wafer location (site):

1. Score all unsampled states using the ensemble model, with a score function that balances explore vs exploit

$$\text{Score}(s) = \mu(s) + \beta \cdot \sigma(s)$$

2. For each optic config group (Polarizer, Aperture, Direction) Select top- k Wavelength and Focus offset candidates with the highest scores.
3. Query the environment to obtain true values for these states.
4. Augment the dataset with the new samples.

5. Retrain the ensemble on the updated dataset.

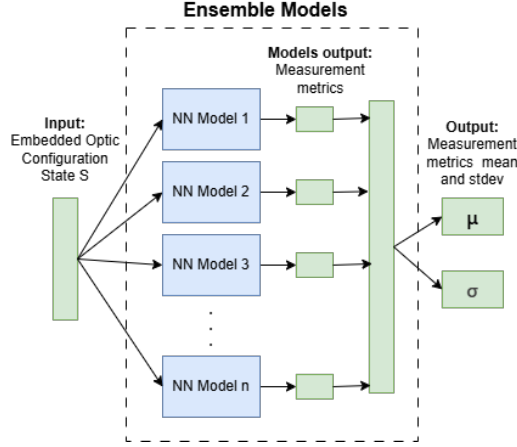


Figure 3: Ensemble-Based Model Uncertainty Estimation.

4 Experimental Setup

4.1 Vanilla Q-Learning

The vanilla Q-Learning was applied on ARO cycle 1 only and follows, the implementation below:

- **Environment setup:** Initiate the initial grid based on the dataset, reset the Q-learning values to zero, and setup the reward distribution.
- **Train:** Regular Q-table updates with several possible hyperparameters
- **Evaluation:** The results were evaluated based on success rate of the agent navigating from 100 random locations on the 2D grid to the goal.

4.2 Model-Based Sampling Optimization

Each ARO wafer recipe dataset, containing 2-3 cycles, will be utilized for both training and evaluation

- **Training:** A subset of samples from Cycle 1 will be used to train the model, as defined in section 3.2
- **Evaluation:** The ensemble model's performance will be evaluated using data from Cycles 2 and 3.
- **Hyperparameter Tuning:** Hyperparameter tuning will be conducted on a single dataset. The optimal hyperparameters obtained will then be applied to train and test models on other datasets.

5 Results

5.1 Vanilla Q-Learning Results

Quantitative Evaluation

The Q-Learning evaluation can be divided into two parts: (1) environment and train algorithm parameters, and (2) typical Q-Learning hyperparameters. For the former, the reader is referred to the methodology section for more details (3.1), for the latter, the tunable hyperparameters are the number of training episodes, discount factor, and learning rate. Truncate parameter to finish the episode after a certain number of unfolds was kept constant at 1000. Additionally, ϵ -greedy to balance between exploration and exploitation was also kept constant at 0.9

The first attempt was to set the reward function of the non-terminal states to -1 ; however, this results in a complete failure where the agent achieves success of only about 12% (Table 1). Then, the reward function of the non-terminal states was $Reward = M - \max(M)$ where M is the weighted KPI values of nearest neighbors. To quantify the robustness of this algorithm, the training phase was conducted 25 times for every configuration and the success criteria was based on the algorithm performance for 100 random starting points (see an illustration of one successful navigation in Figure 5). The metrics to evaluate were the success rate average and standard deviation, and the average steps it takes the agent to navigate from these random 100 starting locations to the highest reward. The results are presented in the table below and include the algorithms of the dynamic reward function and all starting states exploration options:

Table 1: Mean and standard deviation success rate of the agent navigating from 100 random starting points to the reward, and the average number of steps it takes to accomplish it. Hyperparameters were kept constant: learning rate 0.95, discount factor 0.9, ϵ -greedy 0.9, and number of episodes 10,000

All starting locations	Dynamic rewards	Success rate (μ)	Success rate (σ)	number of steps (μ)
True	False	91.88	0.43	18.11
False	False	91.92	0.39	18.15
False	True	15.08	0.07	12.48

From the table above, one can learn that the dynamic reward function doesn't work, and applying the option for exploring all starting locations across episodes doesn't help to improve the results. Noteworthy that when the dynamic reward function succeeds, the average number of steps shows that the algorithm works better if the random starting location is closer to the goal. The "vanilla" option of Q-Learning was thus chosen to exclude the options for exploring all starting locations and dynamic reward function.

Next, to assess whether 92% is the best performance the agent can achieve in this environment, the hyperparameters learning rate, discount factor, and the number of episodes were modified each one at the time while the other two were kept constant at 0.95, 0.90, and 2500. The results are shown in the figure below:

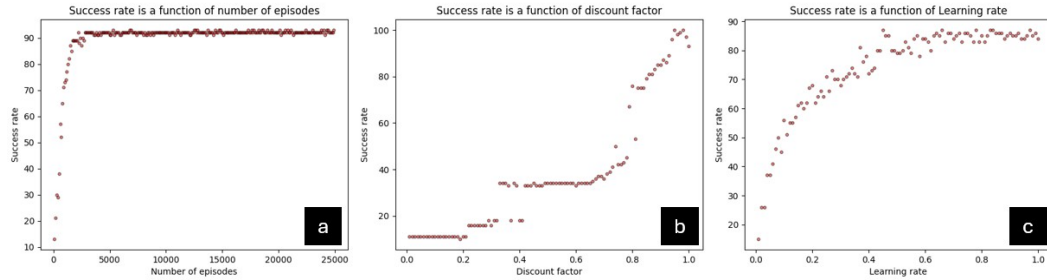


Figure 4: Q-Learning hyperparameters tuning assessed by the success rate of the agent navigating from a random state to the goal as a function of number of episodes (a), discount factor (b), and learning rate (c).

An interesting observation from the Figure 4 (a) is that the agent reaches 100% when the discount factor is somewhere between 0.9 to 1.0; therefore, last attempt for 100 training iterations using a discount factor of 0.95 yields 100% success rate. This observation was replicated on other target types in the dataset and shows equivalent performance.

Qualitative Analysis

Qualitatively investigating the failure of the dynamic reward function shows that for most starting point locations, the agent was trapped in an infinite loop moving up/down or right/left as $Q(s, a)$

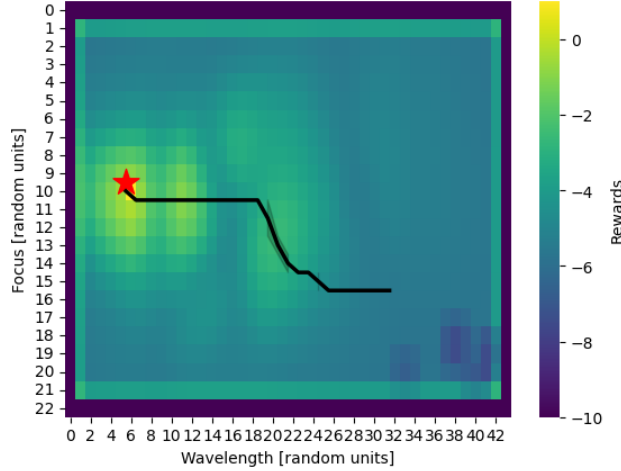


Figure 5: An example of the agent navigates from a random state to the goal.

makes it moving back-and-forth between adjacent states. Additionally, it seems that $Q(s, a)$ are very negative as the agent stepped on the same state more than once in a given episode. The reason why the Q-table looks like that across hundreds of episodes is still not very clear.

5.2 Model-Based Sampling Optimization Results

In this setting, approximately 16% of the samples from Cycle 1 were utilized for training, which constitutes 7.5% of all samples.

Hyperparameter Settings

- **Baseline:** Standard model training configuration, with exploration weight 2.0.
- **Double Weight to New Samples in Retraining:** Increased emphasis on new samples during the retraining phase.
- **Dynamic Exploration Weight:** The exploration parameter, beta, starts at 2.0 and is gradually reduced with each retraining.

The Hyperparameter settings of *Dynamic Exploration Weight* demonstrated slightly improved performance compared to the baseline parameters (Figure 6).

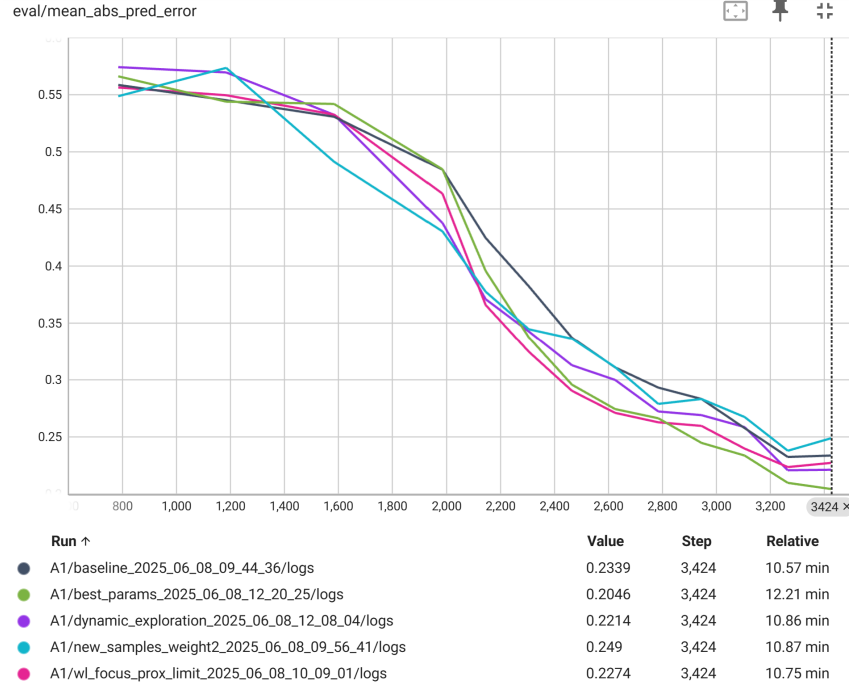


Figure 6: Model-based sampling optimization hyperparameters comparison. The x-axis is the total number of samples used.

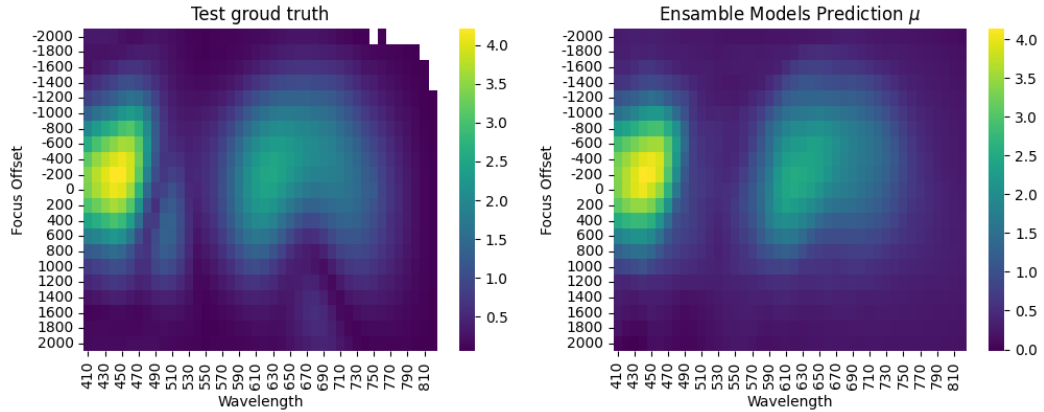


Figure 7: Model predicted value vs ground truth, across wavelength and focus offset for a specific Polarizer and Aperture configuration

Results Across Test sets

The majority of test sets achieved results comparable to those of the validation set, indicating that the algorithm effectively generalizes to new, unseen environments (Figure 8).

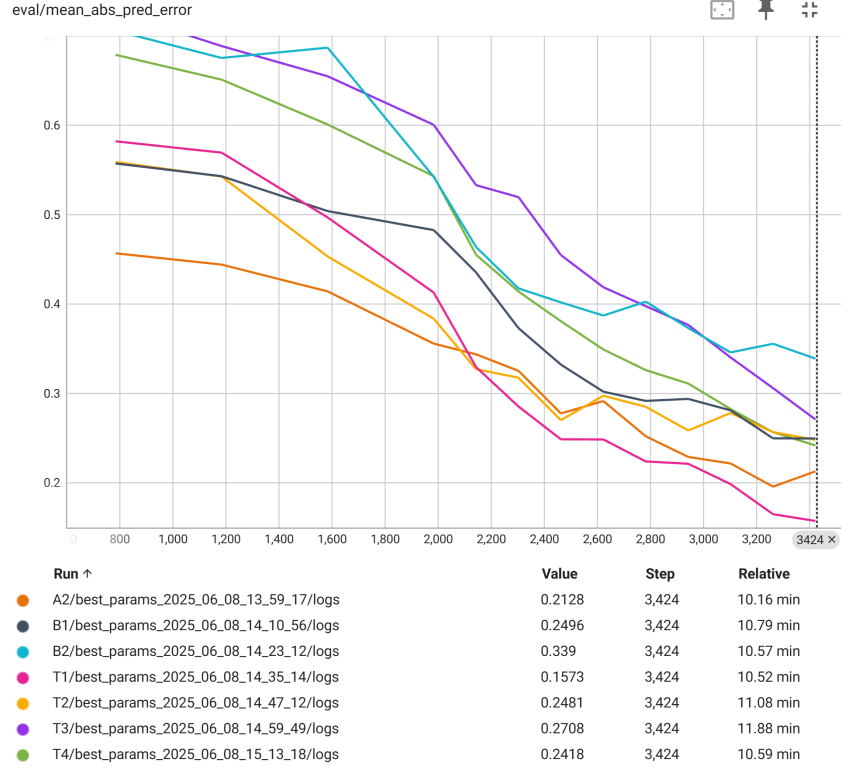


Figure 8: Model-based sampling optimization results across testsets

6 Discussion

6.1 Q-Learning

The fact that the agent reaches 100% success rate though impressive is not that surprising as the Q-table is gradually populated with state-action values that lead the agent to navigate to the goal in the least number of steps. It is interesting to point again that the dynamic reward function has failed in contrast to the authors initial hypothesis; it is challenging to track the behavior of the agent in this case and the underlying reason for this manifestation is not clear yet. The reward function that is constructed as $Reward = M - max(M)$ is much preferable over the constant -1 reward of non-terminal states as it encourages the agent to navigate in a "gradient-ascend" manner and improve its success rate. It is possible that the 2D grid is just too large for a constant reward function to work properly; and thus KPI-driven reward might be preferable in a larger state space. Lastly, a high discount was preferable for the agent to prioritize future rewards rather than immediate ones.

6.2 Sample-based optimization

This algorithm shows good performance and generalization across different datasets. As this algorithm trains neural networks, it is very flexible to network architecture tuning to yield even better results compared to what was shown here. Additionally, it is possible to use several tools to visualize the network parameters in a way that would allow to understand what drives the estimation performance compared to the ground truth.

7 Conclusion

Although Q-learning was able to achieve 100% success rate on several different environments after only a few second of training, is an example of the robustness of the algorithm. An interesting observation was the failure of the algorithm when the reward function on non-terminal states was

homogeneously distributed, which means that the reward function has a crucial rule in the algorithm success. However, this algorithm is not useful for the current project’s objective as the reward function needs to be defined in advance.

Model-based optimization using a Model Ensemble for uncertainty estimation has demonstrated promising results, effectively utilizing only 7.5% of the original data samples. This approach is both generic and robust, as it does not rely on prior knowledge of the configuration landscape, making it applicable to various scenarios.

8 Team Contributions

- **Yatir Linden:**

- Discuss project ideas with colleagues
- Literature review
- Report documentation (proposal, milestone, final report and poster)
- Code implementation
- Q-Learning algorithm implementation

- **Ofer Manos:**

- Discuss project ideas with colleagues
- Setup code environment and software design
- Literature review
- Report documentation (proposal, milestone, final report and poster)
- Code implementation
- Collect and review datasets
- Model-based sampling optimization algorithm implementation

Changes from Proposal No changes were made to the original proposal

References

Zhen-Yin Annie Chen, Chun-Cheng Lin, and Ke-Wen Lu. 2025. Optimizing semiconductor process recipe settings using hybrid meta-learning and metaheuristic approaches. *Information Sciences* 706 (2025), 121998.

Shu-Kai S Fan and Tzu-Jung Chen. 2025. A New Deep Reinforcement Learning Run-to-Run Control Algorithm for Mixed-Product Production Mode in Semiconductor Manufacturing. *IEEE Transactions on Automation Science and Engineering* (2025). <https://ieeexplore.ieee.org/abstract/document/10843296>.

Seung-Soo Han and Gary S May. 1997. Using neural network process models to perform PECVD silicon dioxide recipe synthesis via genetic algorithms. *IEEE transactions on semiconductor manufacturing* 10, 2 (1997), 279–287.

Shlomit Katz, Yoav Grauer, and Efi Megged. 2022. Optical overlay metrology trends in advanced nodes. In *Metrology, Inspection, and Process Control XXXVI*, Vol. 12053. SPIE, 211–228.

Dongsoo Kim, Moran Zaberchik, Chen Li, Honggoo Lee, Chanha Park, Sangho Lee, Dongyoung Lee, Scott Beatty, Jae Y Park, Ramkumar Karur-Shanmugam, et al. 2021. Statistical process optimization method for metrology equipment. In *Metrology, Inspection, and Process Control for Semiconductor Manufacturing XXXV*, Vol. 11611. SPIE, 488–494.

Bangxu Liu, Dewen Zhao, Yuhong Liu, and Xinchun Lu. 2025. Process Control in Semiconductor Manufacturing Based on Deep Distributional Soft Actor–Critic Reinforcement Learning. *IEEE Transactions on Semiconductor Manufacturing* (2025). <https://papers.ssrn.com/sol3/Delivery.cfm?abstractid=4883721>.

Zhu Ma and Tianhong Pan. 2023. Distributional reinforcement learning for run-to-run control in semiconductor manufacturing processes. *Neural Computing and Applications* 35, 26 (2023), 19337–19350. <https://link.springer.com/article/10.1007/s00521-023-08760-1>.

- Gary S May and Costas J Spanos. 2006. *Fundamentals of semiconductor manufacturing and process control*. John Wiley & Sons.
- Boaz Ophir, Udi Shusterman, Anna Golotsvan, Cindy Kato, Masanobu Hayashi, Richika Kato, Tomohiro Goto, Taketo Kuriyama, Manabu Miyake, Yasuki Takeuchi, et al. 2021. Machine learning for Tool Induced Shift (TIS) reduction: an HVM case study. In *Metrology, Inspection, and Process Control for Semiconductor Manufacturing XXXV*, Vol. 11611. SPIE, 23–29.
- Udi Shusterman, Grechin Sveta, Boaz Ophir, Cindy Kato, Masanobu Hayashi, Shengxun Zhao, Jiehong Ng, Tomohiro Goto, Atsushi Imada, Manabu Miyake, et al. 2023. Machine learning robustness in overlay metrology. In *Metrology, Inspection, and Process Control XXXVII*, Vol. 12496. SPIE, 158–167.
- Alexander Verner, Hyunsok Kim, Ikhyun Jeong, Seungwoo Koo, Dongjin Lee, Honggoo Lee, Boaz Ophir, Ohad Bachar, Liran Yerushalmi, Sanghuck Jeon, et al. 2020. Optical overlay measurement accuracy improvement with machine learning. In *Metrology, Inspection, and Process Control for Microlithography XXXIV*, Vol. 11325. SPIE, 325–330.
- Christopher JCH Watkins and Peter Dayan. 1992. Q-learning. *Machine learning* 8 (1992), 279–292.

A Implementation Details

The code was implemented in Python.

Q-Learning algorithm was implemented using only basic packages.

Model-based sampling optimization was implemented using TensorFlow package to build and train the model ensemble.